

BAX 423 Homework 1 Part 1

Logan Garcia, Bonnie Hines

1. Frontier closed model vs open source for a song recommender

Paying for a top vendor model buys speed and fewer headaches. Someone else hosts it and patches it and handles a lot of safety and policy work. A small team does not want to babysit GPUs and giant checkpoints all day. Cost scales with traffic. You are stuck on their roadmap. If they change the model your embeddings and offline jobs can all move at once. You also ship listening data out unless you lock that down in contract.

Open source flips the trade. You host and fine-tune and you own behavior end to end. That helps when cost at scale matters or when compliance says logs stay in your VPC. You still need people who can run the stack and you own every outage and every stale model.

2. Features from listening history and what must update live at serve time

From logs you build things like play counts per artist or genre and recency weighting and skip vs complete and time of day. Maybe a short list of recent track IDs for the last week of listening. Location or device only if the app already uses them.

A playlist that refreshes once a day or week can stay mostly batch. Nightly or hourly jobs write rows to a store and the app reads precomputed results. Nothing has to update every second.

Recommending the next track from what is playing now is different. You need this session. Current track and recent skips or completes and maybe the queue. Those inputs need very low latency so you end up on real-time or near-real-time paths. Daily playlist work is heavy offline jobs plus cache. In-session work is tight SLAs and more calls per user.

3. Daily For-you style playlist vs a recommendation on every new track

Batch playlist work is loose on latency. You can use bigger models and score more items and ship a list people see hours later. If the job fails you retry. You do not block a button.

Per-track recs fire the whole time someone is listening. You need fast inference and a small feature set you can pull quickly and often a smaller model or heavy cache so you do not melt serving. Genre jumps feel bad right away so UX

matters more. Engineering shifts toward online serving and feature stores and rate limits instead of only overnight pipelines.

4. When an artist or subgenre blows up overnight

Watch product metrics first. Clicks and skips and completion split by new tracks and new tags. Not one global accuracy line. A viral spike can leave your old collaborative model wrong before the nightly train lands.

So you shorten how old training data can get and you weight recent listens when you retrain. You add a path for cold tracks with audio or text or similarity clusters so new music is not invisible. Many teams roll models out to a slice of users first. You keep some exploration so the system does not only replay old co-occurrence. A retrain schedule only helps if someone checks how new catalog slices are doing.